



Trajectory Prediction from Hierarchical Perspective

Tangwen Qian
Yongjun Xu
qiantangwen@ict.ac.cn
xyj@ict.ac.cn

Institute of Computing Technology, Chinese Academy of Sciences
University of Chinese Academy of Sciences
Beijing, China

Zhao Zhang
Fei Wang*
zhangzhao2021@ict.ac.cn
wangfei@ict.ac.cn

Institute of Computing Technology, Chinese Academy of Sciences
University of Chinese Academy of Sciences
Beijing, China

ABSTRACT

Predicting the future trajectories of multiple agents is essential for various applications in real life, such as surveillance systems, autonomous driving and social robots. The trajectory prediction task is influenced by many factors, including the individual historical trajectory, interactions between agents and fuzzy nature of an agent's motion. While existing methods have made great progress on the topic of trajectory prediction, they treat all the information uniformly, which limits the sufficiency of using information. To this end, in this paper, we propose to regard all the information in a two-level hierarchical view. Particularly, the first-level view is the inter-trajectory view. In this level, we observe that the difficulty to predict different trajectory samples is different. We define trajectory difficulty and train the proposed model in an "easy-to-hard" schema. The second-level view is the intra-trajectory level. We find the influencing factors for a particular trajectory can be divided into two parts. The first part is global features, which keep stable within a trajectory, i.e., the expected destination. The second part is local features, which change over time, i.e., the current position. We believe that the two types of information should be handled in different ways. The hierarchical view is beneficial to take full advantage of the information in a fine-grained way. Experimental results validate the effectiveness of the proposed model.

CCS CONCEPTS

• Information systems → Spatial-temporal systems; • Computing methodologies → Computer vision.

KEYWORDS

spatial-temporal modeling, trajectory prediction, hierarchical perspective

*Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '22, October 10–14, 2022, Lisboa, Portugal

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9203-7/22/10...\$15.00

<https://doi.org/10.1145/3503161.3548092>

ACM Reference Format:

Tangwen Qian, Yongjun Xu, Zhao Zhang, Fei Wang. 2022. Trajectory Prediction from Hierarchical Perspective. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, October 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3503161.3548092>

1 INTRODUCTION

Predicting the future trajectory of multiple agents is essential for various applications in real life, such as surveillance systems [7], autonomous driving [13] and social robots [33]. Surveillance systems used for city planning must be able to understand how crowds will move and interact to better manage the infrastructure. Autonomous vehicles such as self-driving cars, and social robots such as delivery vehicles are supposed to understand human movement to avoid collisions. In the above applications, accurate trajectory prediction has become a crucial component. However, trajectory prediction is still a challenging task because of several factors, e.g., individual historical trajectory, interactions between agents, and fuzzy nature of the agent's motion [1, 6, 18]. Modeling individual historical trajectory is complicated as trajectory prediction methods should focus on memorizing both spatial and temporal variations in a unified memory pool. Interactions between agents are complex as they contain interactions between not only moving agents, but also moving and stationary agents. To get to the agent's expected destination, there may exist more than one feasible paths that an agent could move along in the future, which is the fuzzy nature of agent's motion.

Recent years have witnessed tremendous progress on the topic of trajectory prediction. For modeling individual historical trajectory, many works modify Recurrent Neural Network (RNN) [27, 31] or Transformer network [5, 17] to seize spatial and temporal information in a unified memory pool. In order to capture interactions between agents, the model architecture is changed from social pooling [1, 6] to graph structure [15, 19, 20] to better capture the complex interactions. To model the fuzzy nature of agent's motion, current works are mostly based on Conditional Generative Adversarial Networks (CGAN) [2, 7] or Conditional Variational Autoencoders (CVAE) [9, 19] to generate more than one single feasible paths. While existing methods have tackled several of the previously listed challenges, they treat all the information uniformly, which hinders the full utilization of information.

In this paper, we propose to treat all the information in a two-level hierarchical view. The first-level view is the inter-trajectory view. In this level, we observe that the difficulty to predict different

trajectory samples is different. As shown in Fig. 1, the purple agent walks with her friend orange agent. At the position where they meet with the stranger blue agent, i.e., the red diamond marks in the blue and purple trajectories, the purple agent changes her path due to the comfortable interactive distance, making the trajectory prediction harder, while the blue agent maintains his nearly straight path upon his narrower comfortable interactive distance. It can be seen that the difficulty to predict the future trajectory of the purple agent is harder than that of the blue agent. However, existing works ignore the difficulty in predicting different trajectories. In order to distinguish between trajectory samples, we define trajectory difficulty and train the proposed model in an “easy-to-hard” schema.

The second-level view is the intra-trajectory view. In this level, we find the influencing factors for a particular trajectory can be divided into two parts, global features and local features. The definition is shown as follows,

- Global features indicate the features that are stable within a trajectory, such as comfortable interactive distance and expected destination of agents.
- Local features denote the features that change over time, such as agents’ positions and their neighbors’ positions.

In Fig. 1, the purple agent and orange agent change their paths due to their own positions and neighbors’ positions, and finally they arrive at the expected destinations, which do not change within a trajectory. Existing works do not explicitly distinguish between two kinds of information, and treat all the information in a uniform way. Different from existing models, we believe that the two types of information should be handled in different ways. The two-level hierarchical view enables our method to take full advantage of all the information in a thorough and fine-grained way, which is beneficial to achieve better performance.

To the best of our knowledge, this is the first work to model trajectory samples’ difficulty, first work to explicitly distinguish between global and local features of a trajectory, and the first work to treat all the information in a hierarchical perspective. Specifically, in this paper, we define a difficulty function to dynamically measure each trajectory sample’s difficulty with the development of model’s capability, then train the model in an “easy-to-hard” schema to adaptively improve model’s learning ability. Furthermore, considering the fact that different features, even inside a trajectory sample, have different influence for future trajectory, we propose local and global feature extraction modules to divide features inside a trajectory into global and local parts, and thoroughly explore relationships between them.

In a nutshell, we highlight our key contributions as follows,

- We propose to treat the information in the trajectory prediction task in a two-level hierarchical view, which are the inter-trajectory view and the intra-trajectory view, respectively.
- In the first level, we define trajectory difficulty and train the proposed model in an “easy-to-hard” schema. In the second level, we distinguish between global and local features, and propose to handle them in different ways.
- Our method achieves the best results on multiple trajectory prediction benchmarks and costs less computation resources than current state-of-the-art works.

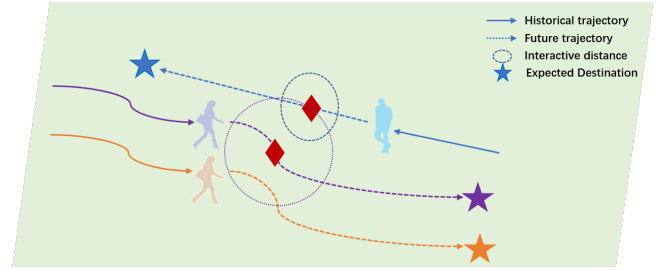


Figure 1: Illustration of the trajectory prediction task. The purple agent walks with her friend orange agent. They meet with the stranger blue agent at the position, i.e., the red diamond marks in the blue and purple trajectories. Agents will consider all neighbors in future movements.

2 RELATED WORK

In this section, we first discuss existing work on the topic of trajectory prediction, then present relevant work that distinguish between different samples and distinguish between features inside a sample.

Research in trajectory prediction can be roughly grouped as modeling individual historical trajectory, interactions between agents, and the fuzzy nature of agent’s motion. For modeling individual historical trajectory, Recurrent Neural Network (RNN) is designed to tackle sequential problems and performs well in time-series modeling. Social LSTM [1] analyzes individual historical trajectory through RNN. However, there are two crucial aspects in individual historical trajectory: spatial correlation and temporal dynamics. Vanilla RNN focuses more on modeling temporal aspect. In order to extract spatial and temporal aspects simultaneously, many works [26, 31] modify RNN to better model spatiotemporal sequence. For instance, PredRNN [26] models spatial and temporal representations in a unified memory cell and conveys the memory both vertically across layers and horizontally over states. However, RNN is still weak and time-consuming in capturing long-range dependency due to its modeling and optimization mechanism. Some works replace RNN with Transformer Network [17] and Bidirectional Transformer (BERT) [5] in trajectory prediction task and perform better.

Interactions between agents are complicated since agents are governed by social norms, such as yielding right-of-way and respecting personal space [4, 8]. Existing algorithms focus on making full use of the interactions between agents. Early works [28] model the interactions by hand-crafted features like several force terms. However, their performance is limited by the quality of manually designed features, and data-driven social pooling-based methods have demonstrated their powerful performance. For example, Social LSTM [1] proposes local social pooling to capture interactions between spatially proximal agents. And Social GAN [6] modifies local social pooling to global social pooling to consider subtle cues for all agents involved in a scene. While social pooling-based works model only the pairwise interactions between agents, which simplifies the interactions between agents. Current works use graph structure to grasp complicated interactions as the topology of graph is a natural way to represent interactions between agents [7, 15].

For instance, StarNet [32] uses a star topology, which includes a unique hub network and multiple host network, to consider the collective influence among agents. Social BiGAT [7] introduces a flexible graph attention network to allow all agents in a scene to interact with each other. RSBG [23] proposes recursive social behavior graph to recursively extract social representations supervised by group-based annotations. Trajectron++ [19] adds physical attention and social attention through graph-structured neighbors.

The agent's motion is fuzzy: given the historical trajectory of the agent, there are many feasible paths that the agent could move along in the future [12, 21]. Most works use a sequence architecture with a latent variable model, such as a Conditional Variational Autoencoder (CVAE) to explicitly encode multimodality [9, 14, 24, 30], or a Conditional Generative Adversarial Network (CGAN) to implicitly do so [2, 6, 7, 15, 18]. For instance, PECNet [14] uses the endpoint estimation VAE for sampling the future endpoints and a trajectory prediction module to use the sampled endpoints to predict future trajectory. Sophie [18] uses the Long Short-Term Memory (LSTM) based GAN module, which takes the highlighted features from designed attention module, to generate a sequence of plausible and realistic future paths for each agent. While existing methods have made great progress in addressing several of the previously listed challenges, they treat all the information uniformly, which limits the sufficiency of using information.

Research in distinguishing between different samples. Recently drawing a distinction between different samples has received growing research interests in natural language processing [22] and computer vision [11]. Researchers design difficulty functions to estimate the overall difficulty of samples, then adopt curriculum learning [3] to enable models to gradually proceed from easy samples to more complex ones in training. To the best of our knowledge, no other work has previously distinguished between different trajectory samples' difficulty in trajectory prediction task. In this paper, we define a trajectory difficulty function to dynamically measure each trajectory sample's difficulty with the development of model's capability.

Research in distinguishing between features inside a sample. Previous works implicitly encode all features into a dense vector and use it to generate future trajectory [1, 6]. Recently some works explicitly extract agent's expected destination from individual historical trajectory and use agent's expected destination as a uniform factor in trajectory generation module [14, 19, 24]. To the best of our knowledge, this paper is the first work to explicitly distinguish between global and local features inside a trajectory sample and thoroughly explore relationships between them.

3 PROBLEM DEFINITION

There exists a time-varying number of interacting agents $1, \dots, N(t)$ in a scene. Here, $N(t)$ is the number of agents at timestamp t . Each agent i has historical trajectory, e.g., its location (x_i^t, y_i^t) in the scene. Therefore, the historical and current trajectories in the scene are represented by the ordered set of multiple agents' locations:

$$X_i^{1:t} = \{(x_i^\tau, y_i^\tau) | \tau = 1, \dots, t\} \quad (1)$$

Here, $\forall i \in [1, \dots, N(t)]$, and t represents the current time instant. Our goal is to predict the future trajectory of multiple agents based

on the historical and current trajectories:

$$Y_i^{t+1:T} = \{(x_i^\tau, y_i^\tau) | \tau = t+1, \dots, T\} \quad (2)$$

4 METHOD

In this paper, we propose a two-level hierarchical view to take full advantage of the information in a fine-grained way for the trajectory prediction task. The first-level view is the inter-trajectory view. In this level, a trajectory difficulty measure module is proposed to define trajectory difficulty to train the proposed model in an "easy-to-hard" schema. The second-level view is the intra-trajectory view. In this level, local feature extraction module, global feature extraction module and trajectory generation module are proposed to distinguish between global and local features in each trajectory sample, and handle them in different ways. The architecture of our method is visualized in Fig. 2. In the following, we provide the details of the above components.

4.1 Inter-Trajectory View

In the view, the trajectory difficulty measure module is proposed. It is used to dynamically measure the difficulty of trajectories, and trains the model with trajectories under the difficulty threshold.

4.1.1 Trajectory Difficulty Measure Module. In this module, we define a dynamic difficulty function to define the difficulty of predicting each trajectory sample. A simpler trajectory has a lower difficulty score. Then we train the proposed model in an "easy-to-hard" schema. We set a difficulty threshold for all the trajectories. The model's training process starts up with a low threshold, i.e., at the beginning, we train the model with easy samples. Then we gradually increase the threshold until taking into account all the samples according to a pacing function. In this way, model's ability in finding the mismatching information is progressively strengthened and will be more robust in real-world scenarios [22].

Dynamic Difficulty Function in our method is defined as below,

$$D(X_i^{1:t}) = \frac{\mathcal{L}_2(Y_i^{t+1:T}, \hat{Y}_i^{t+1:T}) - D_{min}}{D_{max} - D_{min}} \quad (3)$$

Here, $Y_i^{t+1:T}$ and $\hat{Y}_i^{t+1:T}$ represents ground truth and predicted future trajectories respectively. $\mathcal{L}_2$ denotes L2 norm, e.g., euclidean distance between ground truth and the predicted future trajectory. D_{min} and D_{max} denote minimum and maximum L2 norm for all trajectories, used to standardize each trajectory sample's difficulty into the range of $[0, 1]$. 0 means the simplest trajectory, and 1 means the toughest one.

The difficulty function is designed to measure the degree of difficulty for different samples. For example, an agent suddenly changes its next location to avoid conflict with others, would result in a large variation point, which makes trajectory prediction harder. In the above situation, where large variation points exist, the L2 norm tends to be large due to farther distance between $Y_i^{t+1:T}$ and $\hat{Y}_i^{t+1:T}$, resulting in a large value of $D(X_i^{1:t})$. **Besides**, with the enhancement of model's capability during training, the difficulty on trajectory prediction for one certain sample is constantly changing. Therefore, we choose $\hat{Y}_i^{t+1:T}$ as prediction results from last training round to dynamically measure each trajectory sample's difficulty during the training process.

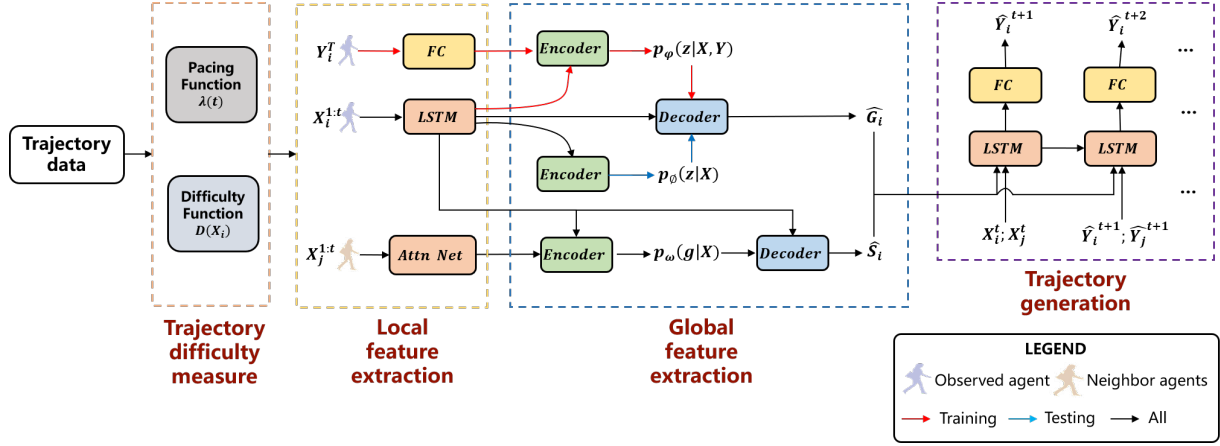


Figure 2: The architecture of our two-level hierarchical method, which are the inter-trajectory view and the intra-trajectory view respectively. It mainly consists of four modules, from left to right, trajectory difficulty measure module, local feature extraction module, global feature extraction module, and trajectory generation module. The leftmost module, i.e., the trajectory difficulty measure module, belongs to the first-level view. The other three modules belong to the second-level view. Red connections are only used in the training stage, blue connections are only used in the evaluation stage, and black connections are used in both stages.

Pacing Function adjusts the training data at every epoch by adjusting the difficulty threshold. The pacing function used in our method is a general root function. [25] showed that the root function is the most beneficial predefined training scheduler for curriculum learning.

$$\lambda(e) = \min(1, \sqrt{\frac{1 - \lambda_0^2}{T_{grow}} \cdot e + \lambda_0^2}) \quad (4)$$

Pacing function $\lambda(e)$ maps training epoch number e to a scalar $\lambda \in (0, 1]$, which means top $\lambda(e)$ proportion easy training examples are available at the e -th epoch. Here λ_0 and T_{grow} are hyper-parameters. λ_0 is the initial proportion at the first epoch. T_{grow} denotes the first T_{grow} epochs in the training stage. At training warm up stage (first T_{grow} epochs), we learn a basic model with an easy subset of the training data. After T_{grow} epochs (i.e., epoch number $e \geq T_{grow}$), $\lambda(e) = 1$ and the model can then freely access the entire training data.

4.2 Intra-Trajectory View

In the view, local feature extraction module, global feature extraction module and trajectory generation module are proposed. The first module encode local features including agent's positions and neighbors' positions. The second module extracts global features including agent's expected destination and agent's comfortable interactive distance. As for the third module, it's fed with local and global features to generate feasible paths.

4.2.1 Local Feature Extraction Module. In our paper, local features are extracted from locations of agents and neighbors. In order to consider the influence of all neighbors, we use aggregation operation and attention mechanism to obtain neighbors' local feature.

For an agent's local feature e_L^A , we use Long Short-Term Memory (LSTM) to model the observed agent's historical trajectory for

its strong performance on sequential summarization tasks [29].

$$e_L^A = LSTM(X_i^{1:t-1}) \quad (5)$$

For neighbors' local feature e_L^N , we use LSTM to model the observed agent's neighbors' historical trajectories firstly, then we aggregate neighbors' feature and the observed agent's current location. Aggregated features are fed into the attention mechanism to obtain the local neighborhood feature, which represents the influence of all neighbors on the observed agent.

$$e_L^N = ATTN([LSTM(X_j^{1:t-1}), f(X_i^t)]) \quad (6)$$

Here, $j \in N(t) \setminus i$. $N(t) \setminus i$ denotes the collection of all $N(t)$ agents excluding the observed agent i . And f represents spatial embedding function to turn current location into a high dimensional vector. Finally, the observed agent's local feature and local neighborhood feature are concatenated to produce local features e_L .

$$e_L = [e_L^A, e_L^N] \quad (7)$$

4.2.2 Global Feature Extraction Module. In our paper, global features consist of agent's expected destination and agent's comfortable interactive distance. The former is extracted from historical trajectory of the observed agent, and the latter is extracted from not only historical trajectory of the observed agent but also historical interactions between the agent and its neighbors.

For agent's expected destination, we adopt the Conditional Variational Autoencoder (CVAE) concepts to obtain its distribution. In this paper, G_i is a high-dimensional vector of an agent's ground truth destination Y_i^T . We encode the expected destination G_i of the observed agent i with a Encoder E_G to represent the observed agent's ground truth destination in neural networks, which is independent for all agents. Here the Encoder E_G is implemented with a multi-layer perception (MLP). The representations are concatenated with the observed agent's local features and passed into the

latent encoder $E_{z,X,Y}$ to learn the distribution of the agent's ground truth destination upon both the observed agent's local features and ground truth destination $p_\psi(z|X, Y)$. Here z represents a latent variable, ψ denotes parameters of the distribution.

$$p_\psi(z|X, Y) \leftarrow E_{z,X,Y}([E_G(Y_i^T), e_L^A]) \quad (8)$$

The part on the left side of $\leftarrow$ is characterized by the part on the right side. We sample possible latent destination $\hat{Y}_i^T$ from the distribution $p_\psi(z|X, Y)$.

$$\hat{Y}_i^T \sim p_\psi(z|X, Y) \quad (9)$$

As the ground truth G_i is unavailable at test time, we also learn the distribution of agent's expected destination $p_\phi(z|X)$ by feeding the latent encoder $E_{z,X}$ only with the observed agent's local features. Here ϕ denotes parameters of the distribution.

$$p_\phi(z|X) \leftarrow E_{z,X}(e_L^A) \quad (10)$$

We expect these two distributions $p_\psi(z|X, Y)$ and $p_\phi(z|X)$ to be as similar as possible. In other words, we expect KL divergence between them to be as small as possible. After that, we sample possible latent destination $\hat{Y}_i^T$ from the distribution $p_\phi(z|X)$ during evaluation phase.

$$\hat{Y}_i^T \sim p_\phi(z|X) \quad (11)$$

After we get sampled latent destination $\hat{Y}_i^T$, we concatenate it with the observed agent's local features and decode the concatenation using a Decoder D_G to yield our prediction for expected destination $\hat{G}_i$. Here the Decoder D_G is implemented with an MLP.

$$\hat{G}_i = D_G([\hat{Y}_i^T, e_L^A]) \quad (12)$$

Global expected destination extraction module is illustrated in Fig. 2. Notice that the red connections are only used in the training phase, blue connections are only used in the evaluation phase, and the black connections are used in both phases.

For agent's comfortable interactive distance, in our end-to-end architecture, it is a hidden representation, which is not human interpretable. The differentiated prediction of each agent indicates differentiated comfortable interactive distance of each agent. we encode neighbors' local feature with a Encoder E_S to represent the observed agent's comfortable interactive distance, which is independent for all agents. Here the Encoder E_S is implemented with an MLP. The representations are concatenated with the observed agent's local features and passed into the latent encoder $E_{z,X}$ to learn the distribution of the latent variable $p_\omega(g|X)$. Here g represents a latent variable, ω denotes parameters of the distribution.

$$p_\omega(g|X) \leftarrow E_{z,X}([E_S(e_L^N), e_L^A]) \quad (13)$$

We sample possible latent comfortable interactive distance S_i from the distribution $p_\omega(g|X)$.

$$S_i \sim p_\omega(g|X) \quad (14)$$

Finally, we concatenate sampled latent comfortable interactive distance with the observed agent's local features and decode the concatenation using a Decoder D_S to yield our prediction for $\hat{S}_i$. Here the Decoder D_S is implemented with an MLP.

$$\hat{S}_i = D_S([S_i, e_L^A]) \quad (15)$$

There are two difference between expected distance extraction and comfortable interactive distance extraction. The first one is

that expected distance is extracted from historical trajectory of the observed agent, and comfortable interactive distance is extracted from not only historical trajectory of the observed agent but also historical interactions between the agent and its neighbors. The second difference is in the evaluation phase, as agent's ground truth destination is available at only training phase, while agent's comfortable interactive distance is available at both phases.

Finally, the observed agent's expected destination and comfortable interactive distance are concatenated to produce the observed agent's global features e_G .

$$e_G = [\hat{G}_i, \hat{S}_i] \quad (16)$$

4.2.3 Trajectory Generation Module. In this module, the observed agent's local features e_L , global features e_G , and current location X_i^t are concatenated and fed into LSTM to generate a location at next timestamp $\hat{Y}_i^{t+1}$.

$$\hat{Y}_i^{t+1} = LSTM([e_L, f(X_i^t), e_G]) \quad (17)$$

Here f is spatial embedding function described in Local Feature Extraction Module.

When predicting the location at $t + 2$ timestamp, agent's local features change into e_L' , current location changes from X_i^t to $\hat{Y}_i^{t+1}$, while agent's global features maintain. They are concatenated into LSTM to generate location at $t + 2$ timestamp.

$$\hat{Y}_i^{t+2} = LSTM([e_L', f(\hat{Y}_i^{t+1}), e_G]) \quad (18)$$

It's worth noticing that global features are maintained while local features change during trajectory generation.

4.3 Loss Function

To train our method, loss function consists of three parts,

$$L = L_{Recon} + L_G + L_{KL} \quad (19)$$

L_{Recon} calculates reconstruction loss between generated trajectories and ground truth. L_G calculates reconstruction loss between ground truth destination G_i and predicted destination $\hat{G}_i$. And L_{KL} calculates KLD regularization loss, which measures how close the sampled distribution is to the distribution of latent variables.

In our method, L_{KL} consists of two parts,

$$L_{KL} = \sum_q L_{KL}(q) + L_{KL}(p_\phi(z|X), p_\psi(z|X, Y)) \quad (20)$$

The first part $L_{KL}(q)$ calculates the KL divergence between prior distribution and q distribution. In the paper, prior distribution is $N(0, 1)$, and $q \in \{p_\psi(z|X, Y), p_\phi(z|X), p_\omega(g|X)\}$.

The second part $L_{KL}(p_\phi(z|X), p_\psi(z|X, Y))$ calculates the KL divergence between $p_\phi(z|X)$ and $p_\psi(z|X, Y)$, as we expect these two distributions to be as similar as possible.

5 EXPERIMENTS

This section gives experimental details firstly. Then, we compare our method's performance against various baselines. We also conduct ablation studies to verify proposed contributions. Finally, we present case studies of our method.

Dataset	Linear	LSTM	S-LSTM	S-GAN-P	S-GAN-V	Ours
ETH	0.84/1.60	0.70/1.45	0.73/1.48	0.60/1.19	0.61/1.22	0.44/0.91
HOTEL	0.35/0.60	0.55/1.17	0.49/1.01	0.52/1.02	0.48/0.95	0.20/0.38
UNIV	0.56/1.01	0.36/0.77	0.41/0.84	0.44/0.84	0.36/0.75	0.20/0.49
ZARA1	0.41/0.74	0.25/0.53	0.27/0.56	0.22/0.43	0.21/0.42	0.16/0.39
ZARA2	0.53/0.95	0.31/0.65	0.33/0.70	0.29/0.58	0.27/0.54	0.13/0.32
AVG	0.54/0.98	0.43/0.91	0.45/0.91	0.41/0.81	0.39/0.78	0.23/0.50

Table 1: Quantitative results for predicting next 8 frames. Error metrics reported are ADE/FDE in meters. Bold indicates the best results.

Dataset	Linear	LSTM	S-LSTM	S-GAN	Sophie	RSBG	Trajectron++	Ours
ETH	1.33/2.94	1.09/2.41	1.09/2.35	0.81/1.52	0.70/1.43	0.80/1.53	0.71/1.68	0.53/1.07
HOTEL	0.39/0.72	0.86/1.91	0.79/1.76	0.72/1.61	0.76/1.67	0.33/0.64	0.22/0.46	0.26/0.50
UNIV	0.82/1.59	0.61/1.31	0.67/1.40	0.60/1.26	0.54/1.24	0.59/1.25	0.41/1.07	0.40/0.96
ZARA1	0.62/1.21	0.41/0.88	0.47/1.00	0.34/0.69	0.30/0.63	0.40/0.86	0.30/0.77	0.29/0.71
ZARA2	0.77/1.48	0.52/1.11	0.56/1.17	0.42/0.84	0.38/0.78	0.30/0.65	0.23/0.59	0.23/0.56
AVG	0.79/1.59	0.70/1.52	0.72/1.54	0.58/1.18	0.54/1.15	0.48/0.99	0.37/0.91	0.34/0.76

Table 2: Quantitative results for predicting next 12 frames. Error metrics reported are ADE/FDE in meters. Bold indicates the best results.

5.1 Experimental Details

In this subsection, we give experimental details of our work, including datasets, baselines, evaluation metrics, and implementation details.

5.1.1 Datasets. We evaluate baselines and our method on commonly used datasets ETH [16] and UCY [10]. These datasets capture real pedestrian trajectories with rich multi-agent interaction scenarios at 2.5 Hz, e.g., 0.4s. They consist of five unique scenes, i.e., ETH-eth, ETH-hotel, UCY-zara1, UCY-zara2, and UCY-univ. Each scene includes top-view images, group annotations, and locations of multi-agents. In our paper, we focus on locations of multi-agents and disregard visual and contextual information, which are not always available in real-world scenes.

5.1.2 Baselines. Our method is compared against the following baselines:

- Linear: a naive model that fits the historical and future trajectories by linear regression;
- LSTM: a classical model that captures the agents' historical trajectories by vanilla LSTM;
- S-LSTM [1]: a discriminative model that combines LSTM with a social pooling layer;
- S-GAN-V and S-GAN-P [6]: two GAN-based models that apply generative modeling to S-LSTM;
- Sophie [18]: a GAN-based model that utilizes attention mechanisms to model both with social and visual attentions;
- RSBG [23]: a model that incorporates the agent context information into learning-based method;
- Trajectron++ [19]: a CVAE-based model that incorporates agent dynamics and semantic maps.

5.1.3 Evaluation Metrics. We employ two commonly used evaluation metrics, minimum ADE/FDE given 1 sample, in trajectory prediction task. Lower results indicate better performances:

Task	Dataset	Trajectron++	Ours
8-12	ETH	220	90
	HOTEL	215	70
	UNIV	160	83
	ZARA1	180	70
	ZARA2	165	40
	AVG	188.0	70.6

Table 3: Training time for 100 epochs (minutes), benchmarked on a computer with a 2080Ti GPU.

- Average Displacement Error (ADE): average Euclidean distance between ground truth locations and predicted locations over all time instants.
- Final Displacement Error (FDE): Euclidean distance between ground truth final location and predicted final location at time instant T .

5.1.4 Implementation Details. During training process, we assume that 8 frames of an agent have been observed. Our goal is to predict next 8 or 12 frames (3.2 or 4.8 seconds). We set a training batch size of 64 and the model is trained for 100 epochs using Adam optimizer. The initial learning rate is 0.001.

5.2 Quantitative Results

Our method is compared with various baselines. The results are illustrated in Tab. 1 and Tab. 2, which include detailed results of average displacement error (ADE) and final displacement error (FDE) in meter space. ADE metric is used to evaluate the deviation about overall trend of predicted future trajectory, while FDE metric describes the deviation about destination of predicted future trajectory.

As seen in Tab. 1 and Tab. 2, statistical results of 8 frames are better than 12 frames, indicating that it's harder to predict farther trajectories into the future. LSTM method performs better than Linear method upon its ability to capture nonlinearity. S-GAN methods achieve better performance than LSTM methods upon its max social pooling mechanism. Sophie method performs better than S-GAN methods as it adds physical attention mechanism to capture visual information of top-view images. RSBG method achieves better performance than S-GAN methods as it incorporates contextual information. Trajectron++ achieves state-of-the-art results recently as it adds physical attention and social attention through graph-structured neighbors.

Overall, our method achieves state-of-the-art performance on both ADE and FDE metrics. All values summarized in Tab. 1 and Tab. 2 are reported by the authors. For predicting next 8 frames, our method reduces the ADE/FDE of S-GAN-V, a better variant of S-GAN than other variants, achieving an increase of 41.03%/35.90% on average. For predicting next 12 frames, our method improve over Trajectron++, one of current state-of-the-art methods, on the average ADE/FDE by 8.11%/16.48%.

It's worth noticing that our method outperforms existing methods even without using visual or contextual information, which are not always available under some circumstances in real-world scenes. It's also worth noting that our method achieves larger improvement on the more challenging task, 8-12, which indicates that the hierarchical view enables our method with a stronger capability for farther trajectory prediction tasks.

We also compare the training time of our method and Trajectron++, one of current state-of-the-art methods. As shown in Tab. 3, it is notable that our method shows an improvement up to 2.6 times on average faster than the current state-of-the-art work Trajectron++, which shows that our method has superiority in training efficiency, indicating less computation resource cost.

5.3 Ablation Results

As shown in Tab. 4, we evaluate various ablations of our method:

- Local: This represents an ablation of our method with only local features;
- C-L: This represents an ablation of our method with local features and utilization of trajectory difficulty function;
- C-LnG: This represents our complete model, which not only consists of global and local features, but also utilizes trajectory difficulty function to train our method.

Local variant performs worst among all variants as it does not treat features differently. C-L variant performs better than Local variant, achieving an increase of average 13.00%/11.50% (ADE/FDE) and 8.54%/7.53% (ADE/FDE) on predicting next 8 and 12 timestamps respectively. The comparison between C-L variant and Local variant proves the effect of inter-trajectory view. C-LnG outperforms C-L variant by average 41.03%/35.06% (ADE/FDE) and 33.33%/26.21% (ADE/FDE) on predicting next 8 and 12 timestamps respectively upon the utilization of intra-trajectory view. The observation indicates that global feature extraction module in intra-trajectory view has even more significant influence over long sequence of trajectory prediction. These two comparisons indicate effectiveness of our two-level hierarchical method.

Task	Dataset	Local	C-L	C-LnG
8-8	ETH	0.63/1.25	0.61/1.21	0.44/0.93
	HOTEL	0.60/1.11	0.50/0.94	0.21/0.42
	UNIV	0.46/0.91	0.34/0.70	0.21/0.49
	ZARA1	0.26/0.52	0.25/0.51	0.16/0.38
	ZARA2	0.29/0.56	0.23/0.47	0.13/0.32
	AVG	0.45/0.87	0.39/0.77	0.23/0.51
8-12	ETH	0.74/1.47	0.70/1.37	0.52/1.17
	HOTEL	0.50/0.98	0.40/0.81	0.28/0.57
	UNIV	0.71/1.40	0.67/1.34	0.40/0.96
	ZARA1	0.44/0.90	0.43/0.88	0.29/0.71
	ZARA2	0.42/0.85	0.37/0.74	0.23/0.56
	AVG	0.56/1.12	0.51/1.03	0.34/0.79

Table 4: Ablation Results. The top part is ablation results for predicting next 8 frames. The bottom part is for predicting next 12 frames. Error metrics reported are ADE/FDE in meters. Bold indicates the best results.

Task	Dataset	C-L	C-LnG
8-8	ETH*	0.60/1.19	0.44/0.91
	HOTEL*	0.62/1.16	0.20/0.38
	UNIV*	0.42/0.81	0.20/0.49
	ZARA1*	0.28/0.55	0.16/0.39
	ZARA2*	0.26/0.51	0.13/0.32
	AVG*	0.44/0.84	0.23/0.50
8-12	ETH*	0.67/1.25	0.53/1.07
	HOTEL*	0.44/0.96	0.26/0.50
	UNIV*	0.74/1.46	0.40/0.96
	ZARA1*	0.43/0.90	0.29/0.71
	ZARA2*	0.38/0.77	0.23/0.56
	AVG*	0.53/1.07	0.34/0.76

Table 5: * indicates adding noise in datasets. The top part is for predicting next 8 frames. The bottom part is for predicting next 12 frames. Error metrics reported are ADE/FDE in meters. Bold indicates the best results.

Considering noise in input data, i.e., agent's and neighbors' trajectories, we take experiments to analyze how noise affects our methods. We add Gaussian noise when modeling agent's and neighbors' historical trajectories. The results are shown in Tab. 5. The results of noisy dataset prediction for C-L are generally worse than that of the original datasets, while with the utilization of global features, the results of noisy dataset prediction for C-LnG are better than that of original datasets. The observation indicates that global features in intra-trajectory view endow models with robustness.

We also conduct ablation experiments to compare easy-to-hard training strategy:

- Random: Make models' training process with randomly selected trajectories;
- Easy-to-Hard: Make models' training process start with simpler trajectories, gradually increasing the difficulty until whole trajectories;

As shown in Tab. 6, random strategy performs worse than easy-to-hard strategy for easy-to-hard strategy fits well with neural



Figure 3: Visualized Predictions. Yellow dots represent the trajectory for past 3.2s while red & cyan dots represent the trajectory of predicted & ground truth future respectively for next 4.8s. ZARA, hotel and ETH scenes are illustrated separately from left to right.

Task	Dataset	Random	Easy-to-Hard
8-8	ETH	0.63/1.25	0.61/1.21
	HOTEL	0.60/1.11	0.50/0.94
	UNIV	0.46/0.91	0.34/0.70
	ZARA1	0.26/0.52	0.25/0.51
	ZARA2	0.29/0.56	0.23/0.47
	AVG	0.45/0.87	0.39/0.77
8-12	ETH	0.74/1.47	0.70/1.37
	HOTEL	0.50/0.98	0.40/0.81
	UNIV	0.71/1.40	0.67/1.34
	ZARA1	0.44/0.90	0.43/0.88
	ZARA2	0.42/0.85	0.37/0.74
	AVG	0.56/1.12	0.51/1.03

Table 6: Impact on different training strategy. Error metrics reported are ADE/FDE in meters. Bold indicates the best results.

models whose competence increase gradually. This result proves the superiority of easy-to-hard strategy.

5.4 Case Studies

In Fig. 3, we represent several visualized prediction results of our method. In the first column (i.e., ZARA scene) of both rows, the ground truth future trajectory almost overlaps with the predicted future trajectory, which shows that our method has the capability of predicting future trajectory which has the same direction and pace with ground truth. In the medium and right columns (i.e., hotel, and ETH scenes), we observe that there is a slight difference between ground truth and predicted trajectory. Comparison above is consistent with quantitative results, that our method performs best in ZARA scene among all scenes.

In the second column of both rows, compared with ground truth (cyan dot), each predicted trajectory point in the future (red dot) has the same direction with smaller pace, which indicates that the velocity of predicted trajectory is slower than the ground truth one.

The slower velocity leads to deviation of predicted trajectory in this scene.

In the third column, compared with ground truth (cyan dot) in first row, each predicted trajectory point in the future (red dot) has the same direction with larger pace, which indicates the velocity of predicted trajectory is faster than ground truth one. And in second row, the velocity of predicted trajectory is slower than ground truth one. Either slower or faster velocity lead to deviation of predicted trajectory in this scene. We plan to capture velocity in our future work for different agents in different scenes to achieve more accurate prediction results.

6 CONCLUSION

In this paper, we present our method in a two-level hierarchical view. In the first view, we define trajectory difficulty and train the proposed model in an “easy-to-hard” schema. In the second view, we distinguish between global and local features, and handle them in different ways. We perform extensive experiments on several benchmarks and our method outperforms the state-of-the-art approaches. Ablation studies are conducted to verify the proposed method. Furthermore, global feature extraction module has an even more significant influence over long sequence prediction and endows our method with robustness in case of noise interference. Case studies indicate that our method predicts future trajectories considering the expected destinations.

In the future, we may consider discovering more information of agents, especially velocity, to better explore the trajectory prediction task. We may also incorporate visual and contextual information for comprehensive multi-modal modeling and further validate our method on other datasets, e.g., Stanford Drone Dataset (SDD) and Trajnet++.

7 ACKNOWLEDGMENTS

This work is partially supported by NSFC under No. 61902376 and No.61902382. Zhao Zhang is supported by the China Postdoctoral Science Foundation under Grant No. 2021M703273.

REFERENCES

- [1] Alexandre Alahi, Kratharth Goel, Vignesh Ramanathan, Alexandre Robicquet, Fei-Fei Li, and Silvio Savarese. 2016. Social LSTM: Human Trajectory Prediction in Crowded Spaces. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 961–971.
- [2] Javad Amirian, Jean-Bernard Hayet, and Julien Pettré. 2019. Social Ways: Learning Multi-Modal Distributions of Pedestrian Trajectories With GANs. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 2964–2972.
- [3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML 2009, Montreal, Quebec, Canada, June 14-18, 2009 (ACM International Conference Proceeding Series, Vol. 382)*. ACM, 41–48. <https://doi.org/10.1145/1553374.1553380>
- [4] Jonathan Carlton, Andy Brown, Caroline Jay, and John Keane. 2021. Using Interaction Data to Predict Engagement with Interactive Media. In *MM '21: ACM Multimedia Conference, Virtual Event, China, October 20 - 24, 2021*. ACM, 1258–1266.
- [5] Francesco Giuliani, Irtiza Hasan, Marco Cristani, and Fabio Galasso. 2020. Transformer Networks for Trajectory Forecasting. *CoRR* abs/2003.08111 (2020). arXiv:2003.08111 <https://arxiv.org/abs/2003.08111>
- [6] Agrim Gupta, Justin Johnson, Li Fei-Fei, Silvio Savarese, and Alexandre Alahi. 2018. Social GAN: Socially Acceptable Trajectories With Generative Adversarial Networks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. IEEE Computer Society, 2255–2264.
- [7] Vineet Kosaraju, Amir Sadeghian, Roberto Martín-Martín, Ian D. Reid, Hamid Rezaatofghi, and Silvio Savarese. 2019. Social-BIGAT: Multimodal Trajectory Forecasting using Bicycle-GAN and Graph Attention Networks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. 137–146.
- [8] Wei-Cheng Lai, Zi-Xiang Xia, Hao-Siang Lin, Lien-Feng Hsu, Hong-Han Shuai, I-Hong Jhuo, and Wen-Huang Cheng. 2020. Trajectory Prediction in Heterogeneous Environment via Attended Ecology Embedding. In *MM '20: The 28th ACM International Conference on Multimedia, Virtual Event / Seattle, WA, USA, October 12-16, 2020*. ACM, 202–210.
- [9] Namhoon Lee, Wongun Choi, Paul Vernaza, Christopher B. Choy, Philip H. S. Torr, and Manmohan Chandraker. 2017. DESIRE: Distant Future Prediction in Dynamic Scenes with Interacting Agents. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 2165–2174. <https://doi.org/10.1109/CVPR.2017.233>
- [10] Alon Lerner, Yiorgos Chrysanthou, and Dani Lischinski. 2007. Crowds by Example. *Comput. Graph. Forum* 26, 3 (2007), 655–664. <https://doi.org/10.1111/j.1467-8659.2007.01089.x>
- [11] Fenglin Liu, Shen Ge, and Xian Wu. 2021. Competence-based Multimodal Curriculum Learning for Medical Report Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*. Association for Computational Linguistics, 3001–3012. <https://doi.org/10.18653/v1/2021.acl-long.234>
- [12] Kedi Lyu, Zhenguang Liu, Shuang Wu, Haipeng Chen, Xuhong Zhang, and Yuyu Yin. 2021. Learning Human Motion Prediction via Stochastic Differential Equations. In *MM '21: ACM Multimedia Conference, Virtual Event, China, October 20 - 24, 2021*. ACM, 4976–4984.
- [13] Yuexin Ma, Xinge Zhu, Sibao Zhang, Ruigang Yang, Wenping Wang, and Dinesh Manocha. 2019. TrafficPredict: Trajectory Prediction for Heterogeneous Traffic Agents. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 6120–6127.
- [14] Karttikeya Mangalam, Harshayu Girase, Shreyas Agarwal, Kuan-Hui Lee, Ehsan Adeli, Jitendra Malik, and Adrien Gaidon. 2020. It Is Not the Journey But the Destination: Endpoint Conditioned Trajectory Prediction. In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12347)*. Springer, 759–776. https://doi.org/10.1007/978-3-030-58536-5_45
- [15] Abdulla Mohamed, Kun Qian, Mohamed Elhoseiny, and Christian G. Claudel. 2020. Social-STGCNN: A Social Spatio-Temporal Graph Convolutional Neural Network for Human Trajectory Prediction. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, 14412–14420.
- [16] Stefano Pellegrini, Andreas Ess, Konrad Schindler, and Luc Van Gool. 2009. You'll never walk alone: Modeling social behavior for multi-target tracking. In *IEEE 12th International Conference on Computer Vision, ICCV 2009, Kyoto, Japan, September 27 - October 4, 2009*. IEEE Computer Society, 261–268. <https://doi.org/10.1109/ICCV.2009.5459260>
- [17] Tangwen Qian, Fei Wang, Yongjun Xu, Yu Jiang, Tao Sun, and Yong Yu. 2020. CABIN: A Novel Cooperative Attention Based Location Prediction Network Using Internal-External Trajectory Dependencies. In *Artificial Neural Networks and Machine Learning - ICANN 2020 - 29th International Conference on Artificial Neural Networks, Bratislava, Slovakia, September 15-18, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12397)*. Springer, 521–532.
- [18] Amir Sadeghian, Vineet Kosaraju, Ali Sadeghian, Noriaki Hirose, Hamid Rezaatofghi, and Silvio Savarese. 2019. SoPhie: An Attentive GAN for Predicting Paths Compliant to Social and Physical Constraints. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 1349–1358. <https://doi.org/10.1109/CVPR.2019.00144>
- [19] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. 2020. Trajectron++: Dynamically-Feasible Trajectory Forecasting with Heterogeneous Data. In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XVIII (Lecture Notes in Computer Science, Vol. 12363)*. Springer, 683–700. https://doi.org/10.1007/978-3-030-58523-5_40
- [20] Zezhi Shao, Zhao Zhang, Fei Wang, and Yongjun Xu. 2022. Pre-training Enhanced Spatial-temporal Graph Neural Network for Multivariate Time Series Forecasting. *CoRR* abs/2206.09113 (2022). <https://doi.org/10.48550/arXiv.2206.09113>
- [21] Pengxiang Su, Zhenguang Liu, Shuang Wu, Lei Zhu, Yifang Yin, and Xuanjing Shen. 2021. Motion Prediction via Joint Dependency Modeling in Phase Space. In *MM '21: ACM Multimedia Conference, Virtual Event, China, October 20 - 24, 2021*. ACM, 713–721. <https://doi.org/10.1145/3474085.3475237>
- [22] Yixuan Su, Deng Cai, Qingyu Zhou, Zibo Lin, Simon Baker, Yunbo Cao, Shuming Shi, Nigel Collier, and Yan Wang. 2021. Dialogue Response Selection with Hierarchical Curriculum Learning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*. Association for Computational Linguistics, 1740–1751. <https://doi.org/10.18653/v1/2021.acl-long.137>
- [23] Jianhua Sun, Qinhong Jiang, and Cewu Lu. 2020. Recursive Social Behavior Graph for Trajectory Prediction. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, 657–666.
- [24] Chuhua Wang, Yuchen Wang, Mingze Xu, and David J. Crandall. 2021. Stepwise Goal-Driven Networks for Trajectory Prediction. *CoRR* abs/2103.14107 (2021). arXiv:2103.14107 <https://arxiv.org/abs/2103.14107>
- [25] Xin Wang, Yudong Chen, and Wenwu Zhu. 2020. A Comprehensive Survey on Curriculum Learning. *CoRR* abs/2010.13166 (2020). arXiv:2010.13166 <https://arxiv.org/abs/2010.13166>
- [26] Yunbo Wang, Haixu Wu, Jianjin Zhang, Zhifeng Gao, Jianmin Wang, Philip S. Yu, and Mingsheng Long. 2021. PredRNN: A Recurrent Neural Network for Spatiotemporal Predictive Learning. *CoRR* abs/2103.09504 (2021). arXiv:2103.09504 <https://arxiv.org/abs/2103.09504>
- [27] Yongjun Xu, Xin Liu, Xin Cao, Changping Huang, Enke Liu, Sen Qian, and Xingchen Liu. 2021. Artificial Intelligence: A Powerful Paradigm for Scientific Research. *The Innovation* 2, 4 (2021).
- [28] Kota Yamaguchi, Alexander C. Berg, Luis E. Ortiz, and Tamara L. Berg. 2011. Who are you with and where are you going?. In *The 24th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2011, Colorado Springs, CO, USA, 20-25 June 2011*. IEEE Computer Society, 1345–1352. <https://doi.org/10.1109/CVPR.2011.5995468>
- [29] Yong Yu, Haina Tang, Fei Wang, Lin Wu, Tangwen Qian, Tao Sun, and Yongjun Xu. 2020. TULSN: Siamese Network for Trajectory-user Linking. In *2020 International Joint Conference on Neural Networks, IJCNN 2020, Glasgow, United Kingdom, July 19-24, 2020*. IEEE, 1–8. <https://doi.org/10.1109/IJCNN48605.2020.9206609>
- [30] Ye Yuan, Xinshuo Weng, Yanglan Ou, and Kris Kitani. 2021. AgentFormer: Agent-Aware Transformers for Socio-Temporal Multi-Agent Forecasting. *CoRR* abs/2103.14023 (2021). arXiv:2103.14023 <https://arxiv.org/abs/2103.14023>
- [31] Pu Zhang, Wanli Ouyang, Pengfei Zhang, Jianru Xue, and Nanning Zheng. 2019. SR-LSTM: State Refinement for LSTM Towards Pedestrian Trajectory Prediction. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 12085–12094.
- [32] Yanliang Zhu, Deheng Qian, Dongchun Ren, and Huaxia Xia. 2019. StarNet: Pedestrian Trajectory Prediction using Deep Neural Network in Star Topology. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2019, Macau, SAR, China, November 3-8, 2019*. IEEE, 8075–8080. <https://doi.org/10.1109/IROS40897.2019.8967811>
- [33] Yanliang Zhu, Dongchun Ren, Mingyu Fan, Deheng Qian, Xin Li, and Huaxia Xia. 2020. Robust Trajectory Forecasting for Multiple Intelligent Agents in Dynamic Scene. *CoRR* abs/2005.13133 (2020). arXiv:2005.13133